

TP 0 Introduction à Python Partie 1

TP Réseaux Avancés M1 SIC - IA

Ilyas Bambrik

Table des matières



Introduction	3
I - Syntaxe Python	5
1. Structure de blocs par tabulations	7
2. Casting de type en python	7
II - Exercice : Commandes système	8
III - Les boucles en python	9
IV - Exercice : Somme des N carrés pairs	11
V - Les tableaux	12
VI - Exercice : Tableaux et Matrices	14
VII - Exercice : Résumé Statistique Partie 1	16
VIII - Les dictionnaires	19
IX - Exercice : Numéro de la chambre	21

Introduction



L'objectif de ce TP est de se familiariser avec la syntaxe de base du langage python.



Python est un langage de script extrêmement simple syntaxiquement et possédant une bibliothèque riche (regex, manipulation des matrices, combinaison et permutation, programmation web coté serveur, etc). Comme d'autres langages de script (php et OTCL), python supporte la programmation orientée objet.

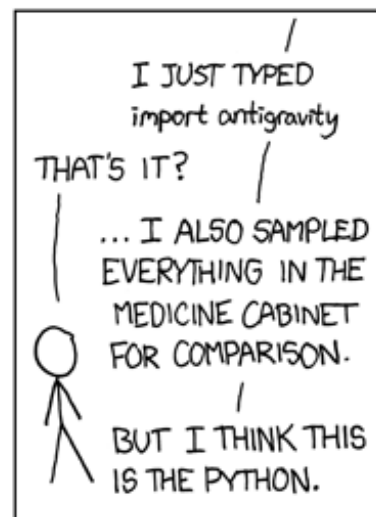
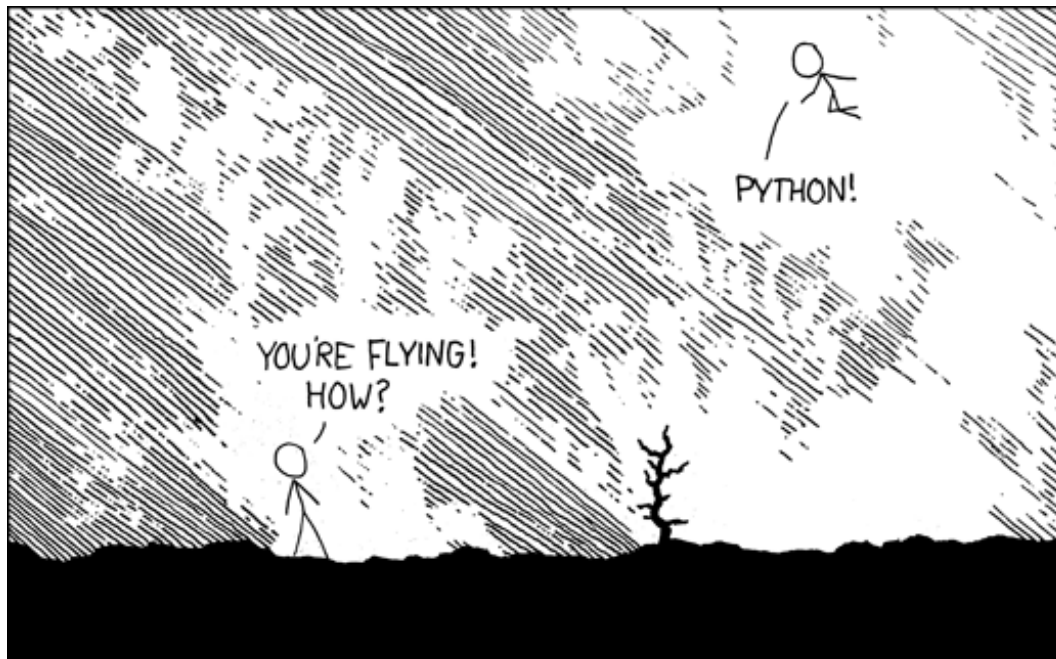
Pendant les dernières années, python a gagné en popularité due aux nombreux projets développés en python comme (par exemple OpenStack, Fsociety et MechanicalSoup).

Afin de vérifier que python est correctement installé, il suffit de vérifier que les fichiers sources python (.py) sont distingués par l'icône python.

Python présente plusieurs avantages :

- Bibliothèque riche;
- Syntaxe simple à apprendre ;
- Prototypage d'algorithme simplifié ;
- Large communauté active dans le développement Web, Intelligence Artificielle, Data Science, Cyber Sécurité et teste de pénétration;





Syntaxe Python

I

Contrairement aux langages *static-typed* comme C et Java, les types des variables sont *dynamiquement* inférés lors de l'affectation. Ceci facilite énormément la programmation ;

Listing 1 Simple programme python

```
1 print('x=')
2 x=int(input())
3 print('p=')
4 p=int(input())
5 print(x, ' puissance ', p, ' =', x**p)
```

- *print* est la fonction d'affichage sur écran comme *printf* et *System.out.print* (et *println*) ;
- *input* permet la lecture d'une ligne de chaîne de caractères comme *scanf* (voir aussi *gets* en C) et *Scanner.next* ;
- Afin de calculer x à la puissance p, l'opérateur **** est utilisé ;

Exemple

Listing 2 Simple calculatrice en python

```
1 #debut de la procedure 'calculatrice'
2
3 def calculatrice(nbr1, nbr2,operation):
4     if (operation == 'A'):
5         return nbr1+nbr2
6     elif (operation == 'S'):
7         return nbr1-nbr2
8     elif (operation == 'M'):
9         return nbr1*nbr2
10    elif (operation == 'D'):
11        if ( nbr2==0):
12            print "Division par zero"
13            return
14        return nbr1/nbr2
15    print("Operation indefini")
16 #fin de la procedure 'calculatrice'
17
18 print("Entrez la valeur de Nbr1=")
19 nbr1=float(input())
20
21 print("Entrez la valeur de Nbr2=")
22 nbr2=float(input())
23
24 print("\nEntrez le type d'operation\nA:\tAddition\nS:\tSoustraction\nM:
    \tMultiplication\nD:\tDivision\n")
25 op=input()
26
```

```
27 print("Resultat= ", calculatrice(nbr1, nbr2,op))
28
```

- Il n'existe pas de commentaire multi-lignes en python et le caractère # marque le début d'un commentaire mono-ligne (comme en TCL et Bash/sh);
- La déclaration d'une procédure (ou fonction) commence par le mot clé *"def"*. Comme dans le corps du programme, les types des paramètres ainsi que le type de retour d'une procédure ne sont pas déclarés explicitement. Par exemple :
def NomProcedure (argument1, argument2,argument3) :
- La clause *"switch"* n'existe pas dans la syntaxe python. Cependant, cette clause est remplacée par *"if-elif-else"*, pour le teste de multiple cas. En outre, une condition est formulée de la manière suivante :
if (condition) :

Par exemple, le programme présenté dans *Listing 3* illustre comment les conditions sont formulées

Listing 3 Les conditions et opérateurs logiques

```
1 def Mention(note):
2     if (note <7):
3         return "F"
4     elif (note >=7 and note<10):
5         return "D"
6     elif (note >=10 and note<13):
7         return "C"
8     elif (note>=13 and note<15):
9         return "B"
10
11     return "A"
12
13
14 Note=int(input())
15 print(Mention(Note))
```

- Le type *"char"* n'existe pas en python. De plus les chaînes de caractères peuvent être enfermer entre *double quotes* (") ou *single quotes* (') comme en *php* et *Javascript*. Une chaîne de caractères multi-lignes est déclarée avec *"""* (ou *'''*) au début et à la fin. Celle-ci peut aussi être utilisée à la place d'un commentaire multi-lignes ;
- Les opérateurs logiques binaires en python sont *and* et *or* à la place *&&* et *||* dans les langages C-type. L'opérateur *not* remplace l'opérateur de négation (!) . En outre, les valeurs logiques en python sont *True* et *False* (sensible à la case) ;

Remarque : Différences entre syntaxe Python 2 et Python 3

Afin de tester les exemples proposés dans ce cours, il faut installer Pytnon 3. En outre, il est important de signaler que la syntaxe de Python 2 est différente de celle de Python 3. Par exemple, la fonction *"raw_input()"* (présente dans python 2.x) est remplacée en python 3.x par la fonction *"input()"* pour la lecture à partir du clavier.

1. Structure de blocs par tabulations

La principale innovation syntaxique en python est l'utilisation de tabulations. La tabulation dans un programme python indique le niveau du bloc et l'imbrication des instructions. Toute violation de cette règle syntaxique est notifiée par un message d'erreur lors de l'interprétation. *Selon le nombre de tabulations séparant l'instruction du début de la ligne, l'interpréteur python infère le bloc où l'instruction est située.* L'imposition de cette convention syntaxique rend la lecture du code source plus facile. Par exemple, voir *Listing 4* :

Listing 4 Structure de bloc en python

```
1 #Bloque~1
2 instruction1
3 instruction2
4 if (Condition1):
5     #Bloque~2
6     instruction3
7     instruction4
8     if (Condition1):
9         #Bloque~3
10        instruction5
11        instruction6
12    instruction7
13 else :
14    #Bloque~4
15    instruction8
16    instruction9
17 instruction10
18 instruction11
```

2. Casting de type en python

Pour effectuer le casting en python, la variable ou la valeur qu'on souhaite convertir est passée en paramètre au type voulu. Le casting en python se fait comme suite :

`Variable1=TypeVariable1(Variable2D_UnAutreType)`

ou bien

`Variable1=TypeVariable1(ValeurD_UnAutreType)`

Cette écriture est équivalente en C / Java à l'instruction suivante :

`Variable1= (TypeVariable1) Variable2D_UnAutreType ;`

`Variable1= (TypeVariable1) ValeurD_UnAutreType ;`

Dans le programme présenté dans *Listing 4* (ligne 28) la valeur lue du clavier (`input()`), qui est une chaîne de caractères, est convertie en type *float* avant d'être affectée à la variable *nbr1*.

Exercice : Commandes système



II

Lisez et exécutez le code suivant permettant à interagir avec le système d'exploitation :

```
1 import os # importe la librerie systeme
2 import time
3 # execute la commande dans explorer / cmd
4 os.system("explorer /")
5 time.sleep(5) # attendre 5 sec
6 # execute la commande explorer http:\\\\www.google.com dans cmd
7 os.system("explorer http:\\\\www.google.com")
8 time.sleep(5) # attendre 5 sec
9 os.system("shutdown /s") # eteindre l'ordinateur
```

Les boucles en python



L'usage de la clause *for* en python est quelque peu atypique. Au lieu de la déclaration traditionnelle, la boucle en python est déclarée comme une itération d'une variable sur un tableau (comme en Bash). Pour l'exemple présenté dans *Listing 5* (ligne 4) la variable "*element*" itère sur les éléments du tableau *tab*. Autrement dit, la variable "*element*" prend la valeur d'un élément de *tab* à chaque itération.

Autres particularités des tableaux en python :

- La déclaration d'un tableau se fait simplement par l'affectation à un tableau vide (ligne 19).
- Les éléments sont dynamiquement ajoutés par la méthode *append* (ligne 22).

Le programme présenté dans *Listing 5* comporte une procédure qui calcule la somme des valeurs des éléments d'un tableau (ligne 2) :

Listing 5 La boucle *for*

```
1 import sys
2 def Somme (tab):
3     valeur_somme=0
4     for element in tab:
5         valeur_somme=valeur_somme+element
6     return valeur_somme
7
8 # Programme principale
9
10 print("Entrez le nombre d'elements du tableau")
11 Nombre_D_Elements=int(input())
12
13 TableauD_Entier=[]
14 for i in range(Nombre_D_Elements):
15     print("TableauD_Entier[" + i + "]=")
16     TableauD_Entier.append(int(input()))
17
18 print("Somme = ", Somme(TableauD_Entier) )
```

La fonction *Somme* suivante est la même fonction présentée dans *Listing 5*, écrite avec la boucle *while* (ligne 4):

Listing 6 La boucle *while*

```
1 def Somme (tab):
2     valeur_somme=0
3     i=0
4     while(i<len(tab)):
5         valeur_somme=valeur_somme+tab[i]
6         i=i+1
7     return valeur_somme
```

L'invocation de la fonction `range(Nombre_D_Element)` (ligne 13- Listing 5) retourne une collection contenant les chiffres allant de 0 jusqu'à `Nombre_D_Element - 1`. Pour voir ce résultat, il suffit d'exécuter le programme suivant (`list` représente le type tableau qui permet de convertir l'objet retourné par `range` en tableau):

```
1 print(list(range(10)))
```

L'affichage de l'exécution précédente sera comme suite :

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

La fonction `range()` peut avoir de 1 jusqu'à 3 paramètres :

`range(N1,N2)` génère un tableau de nombre allant de `N1` jusqu'à `N2-1`

`range(N1,N2,step)` génère un tableau de nombre allant de `N1` jusqu'à `N2-1` un pas de valeur `step`. `Step` peut être négatif

Listing 7 La fonction `range()`

```
1 print("Tableau de 10 a 20",list(range(10,20)))
2
3 print("Tableau de 50 a 100 avec un pas de 15",list(range(50,100,15)))
```

```
Tableau de 10 a 20 [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
Tableau de 50 a 100 avec un pas de 15 [50, 65, 80, 95]
```

Exercice : Somme des N carrés pairs

IV

Question

On vous donne N carrés de tailles croissantes dans l'ordre croissant, et chaque côté de tous les carrés est un nombre pair. Chaque côté du premier carré est 2, et chaque côté de tout carré suivant est plus long que le précédent par 2 unités.

Quelle est la somme des surfaces de tous les N carrés ? Étant donné que le résultat peut être grand, veuillez sortir la réponse modulo 1000000007.

Exemple :

Pour N=3, nous avons 3 carrés de côtés 2, 4 et 6.

La somme des surfaces est $4+16+36 = 56 \% 1000000007 = 56$

Pour N=5, nous avons 5 carrés de côtés 2, 4, 6, 8, 10.

La somme des surfaces est 220 pour N=5.

Input

N : Un entier indiquant le nombre de carrés à additionner.

Output

Un entier indiquant la somme de N carrés modulo 1000000007.

Exemple :

input : 4 => output 120

input : 204 =>output 11402920

Les tableaux



Dans le programme suivant, la taille du tableau (*TailleTableau*, ligne 1) est lue à partir du clavier. Ensuite, les éléments lus à partir du clavier sont ajoutés au tableau (*Tableau.append(Element)*, ligne 5). En outre, la fonction *len()* prend en paramètre un tableau et retourne la taille de celui-ci (ligne 8).

Listing 8 Manipulation des tableaux

```
1 TailleTableau=int(input())
2 Tableau=[]
3 for i in range(TailleTableau):
4     print ("Tableau[",i,"]=")
5     Tableau.append(int(input()))
6 print ("Tableau=",Tableau)
```

```
Taille Tableau=
4
Tableau[ 0 ]=
12
Tableau[ 1 ]=
6
Tableau[ 2 ]=
75
Tableau[ 3 ]=
-2
Tableau= [12, 6, 75, -2]
Taille tableau= 4
```

Comme dans la fonction *range()* présentée précédemment, l'indice de l'élément tableau supporte jusqu'à 3 paramètres (testez l'exemple présenté dans le listing suivant).

- Tableau [*i*] : désigne le i^{em} élément du tableau si ($i \geq 0$). Si $i < 0$, *Tableau [*i*]* désigne l'élément à la position $N-i^{\text{em}}$ avec N définie comme la taille du tableau.
- Tableau [*N1* : *N2*] : désigne la séquence d'éléments du tableau allant de l'indice *N1* jusqu'à *N2-1*.
- Tableau [*N1* : *N2* : *step*] : désigne la séquence d'éléments du tableau allant de l'indice *N1* jusqu'à *N2* avec un pas = *step* (l'élément à l'indice *N2* est exclu de la séquence). Par défaut, Tableau[:] et Tableau[:] sont équivalents à *Tableau[0 :len(Tableau) :1]* (équivalant au tableau entier). La valeur de *step* peut être négative pour parcourir les éléments dans l'ordre inverse :

```
Tab=[1, 3, 5, 7, 9, 11, 13, 15]
```

```
Tab[::-1] # donne [15, 13, 11, 9, 7, 5, 3, 1]
```

```
Tab[ 7: 2: -2] # donne [15, 11, 7]
```

L'objet tableau, représenté par le type *list*, comporte des méthodes prédéfinies permettant de faciliter les traitements sur ces derniers. Soit un objet *Tab* de type *list* :

```
Tab=[7, 3, 1, 2, 10, 5]
```

- `Tab.pop()` : supprime le dernier élément du tableau et retourne celui-ci ;
- `Tab.sort()` : trie les éléments de Tab d'une manière ascendante ;
- `Tab.copy()` : crée une copie de Tab dans un nouveau tableau et retourne celui-ci ;
- `Tab.append(element)` : ajoute element à la fin du tableau ;
- `Tab.index(element)` : retourne la position de la première occurrence de *element* dans Tab. Si celui-ci n'existe pas dans Tab, l'instruction génère une exception *ValueError*;
- `Tab.remove(element)` : supprime la première occurrence de *element* dans Tab. Si celui-ci n'existe pas dans Tab, l'instruction génère une exception *ValueError*;

Pour tester si un élément existe dans un tableau ou pas, l'opérateur `in` est utilisé :

```
1 if (1 in Tab)==True: # ou bien if 1 in Tab:
2     print("Tab contient 1")
3 if (11 in Tab) == False:# ou bien if not(11 in Tab):
4     print("Tab contient 11")
```

Une matrice en python n'est qu'un tableau de tableaux. De la même façon, une matrice de N dimensions peut être représentée.

```
Tab=[[1,2,3],[4,5,6],[7,8,9]]
```

```
print(Tab[1][1]) # donne 5
```

Conseil

La concaténation des listes se fait par une simple opération `+` :

```
Tableau1=[1, 2, 3, 4, 5]
```

```
Tableau2=[6, 7, 8, 9]
```

```
Tableau3=Tableau1+Tableau2
```

```
print(Tableau3)# donne [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

`Tableau2+=[10]` #équivalente à la fonction `append` qui ajoute un élément à la fin avec `append`

Remarque : Type d'éléments d'un tableau

Contrairement à d'autres langages, les éléments d'un tableau python ne doivent pas forcément être du même type. Par exemple, l'écriture suivante est correcte :

```
Tab=[25,"Mohammed","M1INFO",[13.0,12.0,14.0,11.0],12.50]
```

Exercice : Tableaux et Matrices

VI

Question 1

Étant donné une liste d'entiers T de taille $M=N \times N$, donnez une fonction qui prend en paramètre une liste T et N , et retourne une matrice Mat tel que chaque ligne i de Mat ($Mat[i]$) est égale à la liste $[T[i \times N], T[i \times N + 1], T[i \times N + 2], \dots, T[i \times N + N - 1]]$ ($0 \leq i < N$).

Bonus : Votre code doit contenir une seule boucle. Une matrice en Python n'est qu'une liste ou chaque élément est une liste.

Par exemple :

Pour $T=[1, 2, 3, 5, 2, 3, 4, 1, 4, 5, 2, 4, 5, 0, 3, 5]$ et $N=4$

Le résultat attendu est $Mat = [[1, 2, 3, 5], [2, 3, 4, 1], [4, 5, 2, 4], [5, 0, 3, 5]]$ ($Mat[0] = [1, 2, 3, 5]$, $Mat[1] = [2, 3, 4, 1]$, etc).

Question 2

Étant donnée une matrice M de deux dimensions, donnez une fonction qui calcule pour chaque ligne la moyenne de celle-ci (moyenne de la ligne). La fonction doit retourner une liste R , des moyennes tel que $R[i]$ est la moyenne de la i ème ligne de M .

Exemple :

Pour $M = [[1, 2, 3, 5],$

$[2, 3, 4, 1],$

$[4, 5, 2, 4],$

$[5, 0, 3, 5]]$

Le résultat attendu est : $[2.75, 2.5, 3.75, 3.25]$

Question 3

Étant donnée une matrice M carrée de deux dimensions, donnez une fonction qui calcule la somme de ces deux diagonales et retourne cette valeur.

Bonus : Votre fonction doit inclure seulement une seule boucle.

La matrice peut avoir des dimensions pair ou impair.

Exemple :

5	2	-1	3
1	1	2	0
4	3	10	9
7	11	23	5

Pour la matrice prétendante, la somme des diagonales est égale à $5+1+10+5 + 3+2+3+7 = 36$

Exercice : Résumé Statistique Partie 1

VII

Exercice sur les tableaux et boucles

Complétez votre réponse après chaque question. Ne copiez pas le code à partir du pdf, celui-ci est téléchargeable.

```

1 Notes=[15, 13, 5, 11, 5, 14, 9, 6, 14, 1,
2         8, 8, 11, 3, 12, 11, 2, 17, 14, 1, 6, 12, 13, 14, 1]
3 # Question 1 convertir les elements du tableau Notes du type en entier (int) au
  type float.
4 # et imprimer le resultat sur ecran
5 """ le resultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0,
  1.0, 8.0, 8.0, 11.0,
6 3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]"""
7
8
9
10
11 # Question 2 complete le programme pour calculer et imprimer la moyenne sur
  l'ecran
12 # Indication : utiliser le tableau apres conversion des element en float
13 # le resultat attendu = 9.04
14
15
16
17 # Question 3 calculez la variance type de Notes et imprimer le resultat sur ecran
18 # le resultat attendu = 23.6384
19
20
21
22 # Question 4 calculez la valeur la plus frequente dans le tableau et imprimez le
  resultat sur ecran
23 # le resultat attendu = 14.0 ou 14 (se repete 4 fois)
24
25
26 # Question 5 triez le tableau Notes sans utiliser les methodes predefiniees ( .
  sort() et sorted ) dans l'ordre ascendant
27 """le resultat attendu
28 [1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0,
29 11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]
30 ou bien
31 [1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11,
32 11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]
33 """
34
35 # Qustion 6 creez une liste contenant seulement les valeurs uniques de Notes:
36 # Indication : utilisez le tableau apres le trie ou l'operateur 'in' pour tester

```

```

37 # l'appartenance
38 # resultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]
39
40
41

```

Question 1

Convertir les éléments du tableau Notes du type en entier (int) au type float et imprimer le résultat sur écran.

Indice :

Le résultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0, 1.0, 8.0, 8.0, 11.0, 3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]

Question 2

Compléter le programme pour calculer et imprimer la moyenne sur l'écran.

Indice :

Indication : utiliser le tableau après conversion des élément en float

Le résultat attendu = 9.04

Question 3

Calculez la variance de Notes et imprimer le résultat sur écran. ($E(X)$ n'est que la moyenne que vous venez de calculer dans la question précédente).

$$\text{Var}(X) = \frac{\sum_{i=1}^n (X_i - E(X))^2}{n} = E((X - E(X))^2)$$

Indice :

Le résultat attendu = 23.6384

Question 4

Calculer la valeur la plus fréquente dans le tableau et imprimez le résultat sur écran.

Indice :

Le résultat attendu = 14.0 ou 14 (se répète 4 fois)

Question 5

Triez le tableau Notes sans utiliser les méthodes prédéfinies (`.sort()` et `sorted`) dans l'ordre ascendant.

Indice :

Le résultat attendu

[1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0, 11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]

ou bien

[1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11,
11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]

Question 6

Créez une liste contenant seulement les valeurs uniques de Notes.

Indice :

Indication : utilisez le tableau après le trie ou l'opérateur 'in' pour tester l'appartenance.

Résultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]

Les dictionnaires

VIII

Un dictionnaire est une collection qui supporte comme indice une chaîne de caractères (*comme les tableaux associatives en php*) ainsi que d'autres types de valeurs immuables et supportant le hachage :

```
1 UnDictionnaireVide={} # initialisation d'un dictionnaire vide
2 UnDictionnaire ={"Nom":"Mohammed","Specialite":"Informatique","Age":25}
3 print(UnDictionnaire["Specialite"])
4 print(UnDictionnaire.has_key("Adresse"))
5 print(UnDictionnaire.keys())
6 print(UnDictionnaire.values())
```

L'exécution du programme précédant affiche le résultat suivant :

```
Informatique
False
['Nom', 'Age', 'Specialite']
['Mohammed', 25, 'Informatique']
```

La syntaxe de la boucle for-in fonctionne de la même façon sur les dictionnaires. Le code suivant boucle sur les valeurs des clés (keys) avec la variable key (*Ligne 2*) et affiche ensuite chaque clé avec la valeur associée (*Ligne 3*).

```
1 UnDictionnaire ={"Nom":"Mohammed","Specialite":"Informatique","Age":25}
2 for key in UnDictionnaire:
3     print(key , UnDictionnaire[key ])
```

```
1 Nom Mohammed
2 Specialite Informatique
3 Age 25
4
```

Afin de supprimer une clé (key) d'un dictionnaire, la méthode pop est utilisé pour supprimer et retourner la valeur à la clé en paramètre.

```
1 UnDictionnaire ={"Nom":"Mohammed","Specialite":"Informatique","Age":25}
2 print(UnDictionnaire.pop("Nom"))
3 print(UnDictionnaire)
```

```
1 Mohammed
2 {'Specialite': 'Informatique', 'Age': 25}
```

Counter est un type de dictionnaire avec une implémentation permettant de compter le nombre d'occurrences des éléments d'une liste. Le constructeur Counter prend une collections (list, str, etc) et retourne un dictionnaire contenant le nombre d'occurrences de chaque élément de la collection :

```
1 from collections import Counter
```

```
2 Tab=[4,12,5,23,12,4,2,12,4]
3 occurrences=Counter(Tab)
4 print(occurrences)
5 # donne Counter({4: 3, 12: 3, 5: 1, 23: 1, 2: 1})
6 occurrences_caracteres=Counter("""Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6
  2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
7 Type "help", "copyright", "credits" or "license()" for more information.""")
8 print(occurrences_caracteres)
9 #Counter({' ': 22, 'o': 8, '2': 8, '"': 8, 'i': 7, 't': 6, 'n': 6, '.': 6, 'e':
  6, 'r': 6, '4': 5, '3': 4, 'f': 4, 'c': 4, ',': 4, 'y': 3, 'h': 3, '1': 3, '5': 3,
  '(': 3, 'g': 3, 's': 3, ':': 3, '6': 3, '0': 3, ')': 3, 'p': 3, 'a': 2, 'v': 2, 'b':
  2, 'A': 2, 'M': 2, 'l': 2, 'm': 2, 'P': 1, '/': 1, '8': 1, 'u': 1, '7': 1, '[': 1,
  'S': 1, 'C': 1, '9': 1, 'D': 1, ']': 1, 'w': 1, '\n': 1, 'T': 1, 'd': 1})
```

Exercice : Numéro de la chambre

IX

Dans une excursion organisée, chaque famille occupe une chambre et le guide touristique occupe la chambre à lui seul. Pour rapidement inscrire les familles accueillis, le gestionnaire de l'hôtel marque le numéro de la chambre pour chaque personne dans l'excursion, le guide touristique inclus.

Par exemple, si le gestionnaire inscrit la séquence suivante:

1 2 3 6 5 4 4 2 5 3 6 1 6 5 3 2 4 1 2 5 1 4 3 6 8 4 3 1 5 6 2

Cela signifie que la chambre 1 est occupée par une chambre de 5, etc.

Input

La ligne contient les numéros de chambre non ordonnés.

Output

Numéro de la chambre

Exemple :

1 2 3 6 5 4 4 2 5 3 6 1 6 5 3 2 4 1 2 5 1 4 3 6 8 4 3 1 5 6 2

Résultat :

8

Question

Étant donnée une séquence des numéros des chambres réservées pour une excursion écrivez le code pour trouver le numéro de la chambre du guide touristique. Supposez que une famille comporte toujours plus que un membre.

Indice :

Utilisez un dictionnaire pour trouver le numéro de la chambre que se répète une seule fois. Ou bien, vous pouvez aussi utiliser la méthode `.sort()` pour faciliter la recherche de la valeur unique.